Report

engineer · 1 sessions · Jul 17, 2026

You ship wide data systems fast, steer Claude with precise specs, and need tighter proof after big architecture calls.

Your narrative

You delivered a strong end-to-end data engineering challenge with clear task direction, fast Claude Code execution, and good oversight, while leaving the biggest growth opportunity in closing the loop after high-value technical decisions.

What You Built

You completed a Kapital medium data engineering challenge end to end.

The work included synthetic data generation with Python/Faker, validation artifacts, failed-record reporting, dimensional modeling using a dbt-like runner, KPI SQL/reporting, README updates, Docker Compose infrastructure, loaded star-schema tables, and a Grafana KPI dashboard bonus.

You used Claude Code across the analyzed session.

Decision Patterns

Your decision style is architecture-heavy and outcome-focused: 6 of 8 decisions were architecture decisions, and 7 of 8 were high-value decisions. You are good at identifying what matters, especially system shape, data flow, infrastructure, and correctness risks.

Two patterns stood out. **High Standards, Low Closure Rate** means you raised the right bar, but 0 of 8 tracked outcomes were positive, so the final verification did not consistently match the quality of your asks. **Architecture Dominates The Decision Surface** means most of your judgment went into structure and delivery infrastructure, while product and tooling each got only 1 of 8 decisions.

Strengths

You use Claude Code effectively to turn a large assignment into shipped artifacts. In 1 of your 1 sessions, you moved from reading challenge_medium_data_engineer.md through generation, validation, modeling, KPIs, documentation, Docker Compose, and Grafana.

You set concrete requirements instead of vague prompts. Date-format validation, Excel and Markdown reports, the failed-record CSV columns, the KPI list, and the required SQL constructs gave the agent a clear target.

You showed real technical oversight. You caught that Docker/Grafana started but had not loaded data, and you flagged unrealistic average ticket amounts in Grafana as a likely formatting, scaling, or data generation issue.

Growth Areas

Tighten closure after high-value decisions. You made 8 detected decisions and 7 were high-value, but 0 of 8 tracked outcomes were positive; for each major decision, add a final acceptance check like “prove the dashboard reads loaded star-schema data” or “show the KPI output with sane units.”

When you choose a core tool, verify that the implementation still matches the stated requirement after a workaround. You asked for dbt, but after dbt-duckdb failed, the result became a lightweight dbt-like runner with scripts/dbt_runner.py; next time, decide explicitly whether that substitution is acceptable or whether real dbt CLI compatibility is required.

Make the plan-before-coding step more explicit in longer sessions. This upload had 0 planned shipping sessions in 1 of your 1 sessions, so the next useful move is to write a short acceptance plan before implementation, then check each item before calling the work done.

How you use AI

Dances with Robots

Riffs with the AI like a jam session.
Ideas bounce back and forth.

3 evidence points